



Higher Diploma in Computer Science

# Computer Systems & Networks



# Week 2 Class 1 LAB SOLUTIONS

From your **01. Wildcards** lab

- Using find to return files that begin with **temp**

```
compsys@compsys-virtualbox:~$ find ~ -name "temp*"
/home/compsys/temp
/home/compsys/snap/firefox/common/.mozilla/firefox/o49vw619.default/storage/temporary
/home/compsys/tempdate
compsys@compsys-virtualbox:~$
```

# Week 2 Class 1 LAB SOLUTIONS

## From your **03. IO Redirection** lab

Use redirection to allow the user to add ( ``echo`` ) a new record details from the command line and it will be appended to the ``employee.txt`` file.

```
compsys@compsys-virtualbox:~$ echo "123 Caroline Manager Sales €1234" >> employee.txt
compsys@compsys-virtualbox:~$ cat emp*
300 frank manager sales €5000
400 caroline developer technology €5500
999 catherine dev tech €4500
100 mary sysadmin technology €7000
200 rosanne manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
echo "There are no $ currency, only € in employee.txt"
just another example katherine
666 Katherine dev tech €6000
123 Caroline Manager Sales €1234
```

# Sorting Files or Input

↓ Z   ↓ A  
↓ A   ↓ Z

↓ 9   ↓ 1  
↓ 1   ↓ 9

# Sorting Files for Input

- The `sort` command can be used to rearrange the lines of files or input in either dictionary or numeric order.

`$ sort filename`

- `sort` command can rearrange the output based on the contents of one or more fields

`$ sort employee.txt`

## Delimited Files

- Fields are determined by a *field delimiter* contained on each line
  - `tab` character is the default delimiter
  - REMEMBER a "space" in between each word is actually a *single tab character*

# Fields and Sort Options

- the following command can be used to sort the `passwd` file (alphabetically by first column by default)

```
compsys@compsys-virtualbox:~$ sort -t: passwd
```

- `-t` option will allow for another field separator to be specified
- You can specify to sort by a specific field (i.e. column 3) in numeric order:

```
lbox:~$ sort -t: -n -k3 passwd
```

- `-n` option is used to perform a numeric sort.
- `-k` option allowed you specify which field to `sort` by



# Viewing File Statistics

# Viewing File Statistics

- The `wc` command provides the number of lines, words and bytes (1 byte = 1 character in a text file) for a file, and a total line count if more than one file is specified.

```
$ wc employee.txt
```

- It is also possible to view only specific statistics:
  - Use the `-l` option to show just the number of lines
  - The `-w` option to show just the number of words
  - The `-c` option to show just the number of bytes, or any combination of these options

```
ubuntu@golinux:~$ cat fruits.txt
apple
banana
mango
orange
watermelon
ubuntu@golinux:~$ cut -b 1 fruits.txt
a
b
m
o
w
```

# Filtering Files

# Filter File Sections

- The `cut` command can extract columns of text from (i) a file or (ii) standard input.
- The `cut` command is used for working with delimited files

```
lbox:~$ cut -d: -f1,5-7 passwd
```

- `-d` option allows a specific alternative delimiter (default in `cut` expects input to be separated by the **Tab** character)
- `-f` option can specify which fields to display (here it displays the first, fifth, sixth and seventh fields from a `passwd` file)

```
albox:~$ cut -d: -f1,5-7 passwd | head -3
```

# Filter File Sections

`-c` option is used to extract columns of text based upon character position

Return only the first three characters of the *employee.txt* file:

```
oX:~$ cut -c1-3 empl*
```

(globally search  
for a regular  
expression and  
**print** matching  
lines)



**grep**

# grep

on every distribution  
of Linux

search directories and  
files for keywords and  
strings

- search input files for a search string, and print the lines that match it

is a pattern searcher

- If the pattern is found, it displays the line containing it.

**\$ grep *string filename***

# grep

One of the renowned  
search tool on Unix-  
like systems

can be used to  
search for anything

- a file
- or a line
- or multiple lines

**\$ grep *string filename***

# Different Versions

```
graph TD; A[Different Versions] --- B[grep]; A --- C[grep -E]; A --- D[grep -F];
```

grep

grep

grep -E

grep -F

# Syntax

```
compsys@compsys-virtualbox:~$ grep "sysadmin" employee.txt
100 mary sysadmin technology €7000
600 colm sysadmin technology €4500
```

- `-n` parameter to show line numbers

```
3:100 mary sysadmin technology €7000
6:600 colm sysadmin technology €4500
```

- `-v` prints all of the lines that do not match the search string

```
1:300 frank manager sales €5000
2:400 caroline developer technology €5500
4:200 rosanne manager marketing €9500
5:500 eamonn dba technology €6000
```

# Syntax

- **-c** option tells grep to suppress the printing of matching lines, and only display the number of lines that match (or **-v** for *do not* match) the query.

```
compsys@compsys-virtualbox:~$ grep -c "sysadmin" employee.txt
2
compsys@compsys-virtualbox:~$ grep -cv "sysadmin" employee.txt
4
```

# Meta characters of grep

- + Equivalent to one or more occurrences of previous character.
- ? almost 1 repetition of previous character (**a?** match 'a' or 'aa')
- ( Start of alternation expression
- ) End of alternation expression
- | Matching either of the expression separated by '|'.  
“(a|b)cde” would match either ‘acde’ or ‘bcde’
- { start of range specifier.  
“a{2}” matches “aa” in file i.e. the letter ‘a’ 2 times.
- } end of range specifier

# Using metacharacters **egrep**

^ → carat metacharacter indicated the beginning of the line

```
compsys@compsys-virtualbox:~$ grep '^4' employ*  
400 caroline developer technology €5500
```

\$ metacharacter indicates the end of the line

```
y €4500  
~$ grep '4$' employ*
```

So then, what will this search for?

```
$ grep '^4$' employ*
```

# Using Regular Expressions With **grep**

- → zero or more instances of characters matching the pattern
  - You want to search for all instances of the letters **ar** in employee.txt

```
~$ grep 'ar*' employ*
```

- This command reads “match all lines that start with “a” and may or may not have in them extra “r”

```
$ grep 'arr*' employ*
```

# grep as a filter

- as a filter with other commands
- “|” pipe the output of the command through grep.

```
~$ ls -l | grep '^d'
```

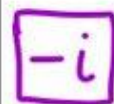
# ♥ grep ♥

JULIA EVANS  
@b0rk

grep lets you search  
files for text

```
$ grep bananas foo.txt
```

Here are some of my  
favourite grep command  
line arguments!

 **-i** case insensitive

 **-A** Show context for your  
search.

 **-B**  
 **-C**  
\$ grep -A 3 foo  
will show 3 lines of  
context after a match

 **-E**  
aka  
egrep

Use if you want  
regexps like ".+" to  
work. otherwise you  
need to use ".\+"

 **-v**

invert match: find  
all lines that don't  
match

 **-l**

only show the  
filenames of the files  
that matched

 **-F**

don't treat the match  
string as a regex  
eg \$ grep -F ...

 **-r**

recursive! Search  
all the files in  
a directory.

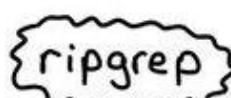
 **-o**

only print the  
matching part of  
the line (not the  
whole line)

 **-a**

search binaries:  
treat binary data  
like it's text instead  
of ignoring it!

grep alternatives

 ack  ag  ripgrep  
(better for searching code!)